

Proyecto de Software

Introducción a Flask

Temario

- Desarrollo del lado del servidor.
- Frameworks.
- Flask.
- Request / Response.
- Rutas.
- Templates: introducción.
- Organización inicial de una aplicación.

Objetivos de hoy

Al terminar la clase deberíamos poder:

- explicar el recorrido de un request;
- asociar una URL con una función;
- generar una response;
- reconocer cuándo usar un template;
- ejecutar y diagnosticar una aplicación pequeña.

No necesitamos memorizar Flask: necesitamos comprender el flujo.

Desarrollo web

Una aplicación web involucra, como mínimo:

```
cliente
  ↓ request
servidor
  ↓
aplicación
  ↓ response
cliente
```

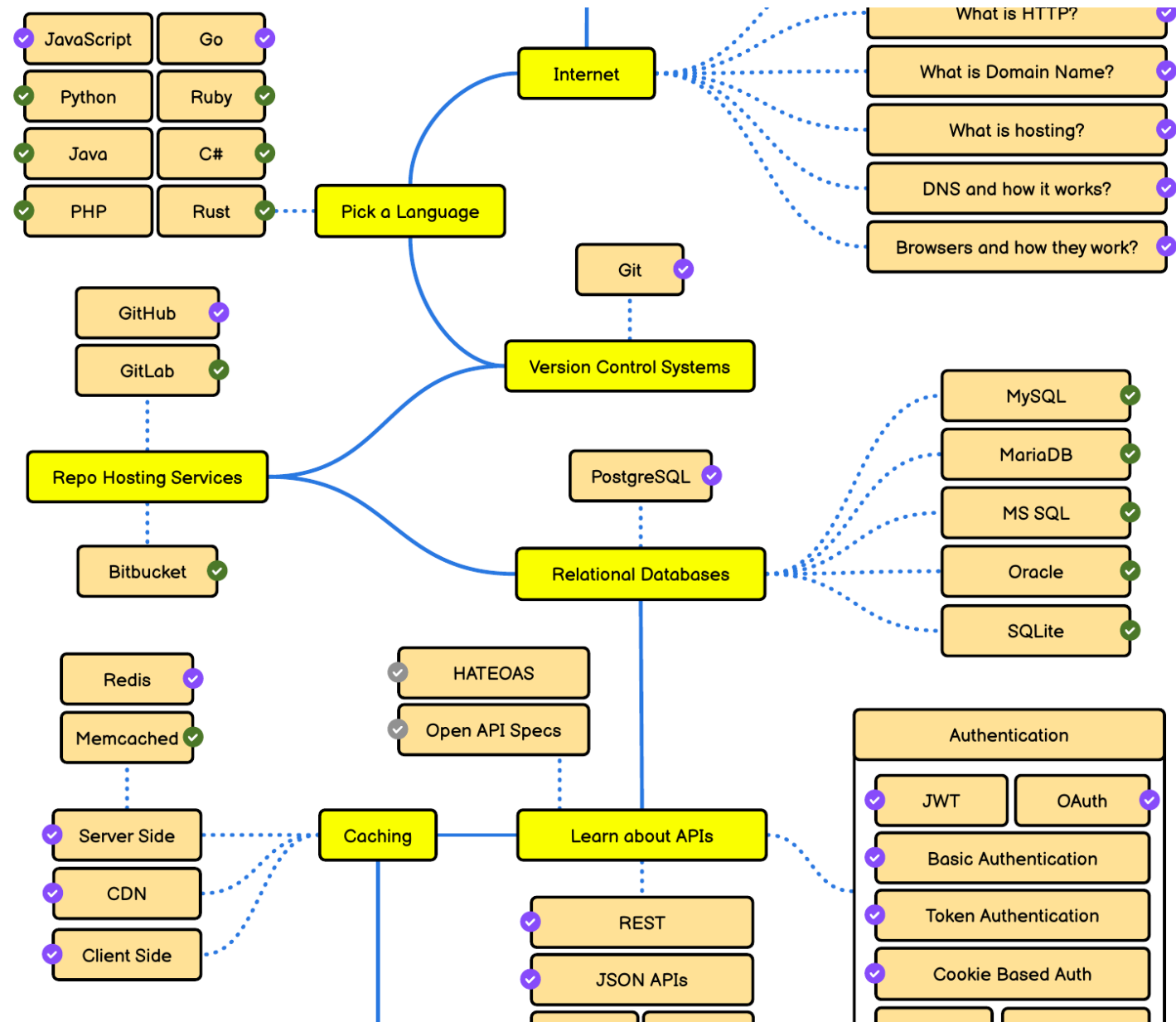
Hoy nos vamos a concentrar en el **servidor**.

Backend: un mapa, no una lista

No intentaremos leer cada nodo.

El roadmap nos ayuda a:

- reconocer la amplitud del backend;
- relacionar conceptos;
- elegir un recorrido posible.



Nuestro recorrido

El roadmap muestra muchas posibilidades. En la materia vamos a construir un camino:

```
Python → Flask → HTTP → PostgreSQL → ORM  
          → REST → seguridad → autenticación
```

No intentaremos aprender todas las tecnologías al mismo tiempo.

Hoy damos el primer paso: una aplicación Flask que recibe un request y genera una response.

Programación en el servidor

El servidor recibe solicitudes HTTP y genera respuestas.

Una aplicación puede:

- procesar parámetros;
- consultar datos;
- aplicar reglas;
- generar HTML;
- devolver JSON;
- responder errores.

Nosotros usaremos Python

Python será nuestro lenguaje del lado del servidor.

No porque sea la única alternativa.

Podríamos desarrollar backend con:

- Java;
- JavaScript/TypeScript;
- PHP;
- Ruby;
- Go;
- C#;
- otros.

¿Qué es un framework?

Un framework proporciona una estructura y herramientas para resolver problemas comunes.

Nos ayuda a:

- evitar código repetitivo;
- organizar la aplicación;
- usar convenciones;
- reutilizar componentes.

Framework ≠ aplicación

Un framework nos da piezas.

Nosotros seguimos tomando decisiones sobre:

- estructura;
- responsabilidades;
- validaciones;
- persistencia;
- seguridad;
- comportamiento.

Flask

Flask es un framework web para Python.

Nos permite, entre otras cosas:

- asociar URLs con funciones;
- recibir requests;
- devolver responses;
- renderizar templates;
- organizar una aplicación modularmente.

Nuestra primera aplicación

```
from flask import Flask  
  
app = Flask(__name__)  
  
@app.route("/")  
def home():  
    return "Hola mundo"
```

¿Qué está ocurriendo?

Cuando el navegador solicita:

```
GET /
```

Flask busca qué función atiende esa ruta.

```
@app.route("/")  
def home():  
    return "Hola mundo"
```

La función genera la respuesta.

El flujo

```
Browser
  |
  | GET /
  v
Flask
  |
  | busca la ruta
  v
home()
  |
  | genera respuesta
  v
Browser
```

Levantamos el servidor

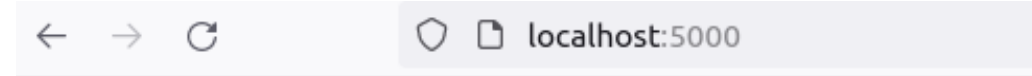
Con el entorno administrado por Poetry:

```
poetry run flask --app app run --debug
```

Durante desarrollo, el modo debug facilita:

- recarga;
- mensajes de error;
- diagnóstico.

El modo debug no es una configuración de producción.



Hola mundo!

Cuando algo sale mal

Supongamos que modificamos la ruta:

```
@app.route("/")  
def home():  
    cantidad = 2  
    return "Cantidad: " + cantidad
```

La aplicación encuentra la ruta, pero no puede generar la respuesta.

Equivocarse es parte del ciclo de desarrollo.

Leer el error

Ante una excepción buscamos:

1. **Tipo:** ¿qué clase de error ocurrió?
2. **Mensaje:** ¿qué operación no pudo realizarse?
3. **Archivo y línea:** ¿dónde aparece el problema?
4. **Contexto:** ¿qué valores participaban?

```
TypeError: can only concatenate str (not "int") to str
```

El traceback se lee desde la operación que falló hacia el origen del flujo.

El debugger permite ejecutar código: sólo debe habilitarse en desarrollo.

Una nueva ruta

Tomamos una idea del integrador, todavía muy simplificada:

```
@app.route("/solicitudes")  
def solicitudes():  
    return "Solicitudes"
```

Ahora existen dos recursos:

```
/
```

```
/solicitudes
```

Parámetros en la URL

```
@app.route("/solicitudes/<int:solicitud_id>")  
def solicitud_detalle(solicitud_id):  
    return f"Solicitud {solicitud_id}"
```

Por ejemplo:

```
/solicitudes/1  
/solicitudes/42
```

Checkpoint: rutas

Tenemos estas dos funciones:

```
@app.route("/solicitudes")  
def solicitudes(): ...  
  
@app.route("/solicitudes/<int:solicitud_id>")  
def solicitud_detalle(solicitud_id): ...
```

¿Qué función se ejecuta para cada request?

```
GET /solicitudes  
GET /solicitudes/42  
GET /solicitudes/abc
```

Request

Una solicitud HTTP puede incluir información en:

- URL;
- query string;
- headers;
- cookies;
- body.

Flask la representa mediante el objeto `request` .

Response

Nuestra función no "imprime una página".

Genera una **respuesta HTTP**.

Puede contener:

- cuerpo;
- código de estado;
- headers.

Ejemplos:

```
200 OK
404 Not Found
500 Internal Server Error
```

Una primera tentación

Podríamos generar HTML desde Python:

```
@app.route("/")
def home():
    html = "<html>"
    html += "<body>"
    html += "<h1>Hola mundo</h1>"
    html += "</body>"
    html += "</html>"
    return html
```

Funciona.

¿Entonces cuál es el problema?

Si la aplicación crece:

```
@app.route("/")  
def home():  
    # lógica  
    # validaciones  
    # HTML  
    # datos  
    # estilos  
    # ...
```

¿Queremos que todo termine en el mismo lugar?

Separar presentación

Flask puede renderizar templates:

```
from flask import render_template

@app.route("/")
def home():
    return render_template("index.html")
```

templates/index.html

```
<!DOCTYPE html>
<html lang="es">
  <head>
    <title>Proyecto de Software</title>
  </head>
  <body>
    <h1>Hola mundo</h1>
  </body>
</html>
```

Datos dinámicos

```
@app.route("/solicitudes/<int:solicitud_id>")
def solicitud_detalle(solicitud_id):
    return render_template(
        "solicitud.html",
        solicitud_id=solicitud_id
    )
```

Y en el template:

```
<h1>Solicitud {{ solicitud_id }}</h1>
```

Una idea importante

Ahora tenemos, al menos, dos responsabilidades:

Python

- recibe request
- decide qué hacer
- prepara datos

Template

- presenta esos datos

Esta separación será central en la próxima clase.

Mostrar el formulario

Una ruta de interfaz puede mostrar el formulario vacío:

```
@app.route("/solicitudes/nueva")  
def solicitud_formulario():  
    return render_template("solicitud_nueva.html")
```

Todavía no se creó ninguna solicitud.

Formularios

Una página puede enviar información al servidor:

```
<form action="/solicitudes" method="post">  
  <input name="cantidad" type="number">  
  <button>Enviar</button>  
</form>
```

Recibiendo datos

```
from flask import request

@app.route("/solicitudes", methods=["POST"])
def solicitud_crear():
    cantidad = request.form.get("cantidad")
    return f"Cantidad recibida: {cantidad}"
```

/solicitudes/nueva muestra la interfaz. POST /solicitudes expresa la creación.

GET y POST

Simplificando:

GET

Se utiliza principalmente para obtener recursos.

POST

Se utiliza para enviar información que produce una acción o creación en el servidor.

Más adelante veremos HTTP y REST con mayor profundidad.

Checkpoint: seguir el flujo

Para este request:

```
POST /solicitudes
```

¿Podemos señalar dónde ocurre cada paso?

```
navegador → Flask → ruta → función  
                → datos del formulario  
                → response → navegador
```

¿Qué cambiaría si la ruta sólo aceptara `GET` ?

La aplicación empieza a crecer

Podemos tener:

```
/
/solicitudes
/solicitudes/42
/solicitudes/nueva
/login
/admin
...
```

Y además aparecen templates, configuración, acceso a datos, validaciones y reglas de negocio.

¿Todo en app.py?

Para una demostración:

```
app.py
```

puede alcanzar.

Para una aplicación mayor necesitamos empezar a separar componentes.

Application Factory

Un patrón habitual en Flask, que retomaremos al estudiar MVC y Blueprints:

```
from flask import Flask

def create_app():
    app = Flask(__name__)

    @app.route("/")
    def home():
        return "Hola"

    return app
```

Nos permite centralizar la creación y configuración de la aplicación.

Por ahora importa reconocer el problema que resuelve, no memorizar el patrón.

Una posible estructura inicial

```
proyecto/  
├── app.py  
├── src/  
│   ├── core/  
│   └── web/  
│       ├── templates/  
│       └── static/  
└── pyproject.toml
```

No existe una única estructura obligatoria.

La estructura debe ayudarnos a comprender y mantener el sistema.

¿Qué debería quedar en cada lugar?

Todavía no tenemos la respuesta completa.

En las próximas clases veremos cómo separar:

- presentación;
- lógica de aplicación;
- acceso a datos;
- configuración.

Eso nos lleva a hablar de **arquitectura y MVC**.

Código que funciona $\neq$ buen diseño

Supongamos que una aplicación:

- responde correctamente;
- guarda datos;
- pasa una demo.

¿Eso alcanza?

Es necesario, pero no suficiente.

También necesitamos una aplicación **comprensible, modificable, testeable, mantenible y explicable.**

¿Y si el código lo genera un agente?

Un agente puede crear rutas, formularios, templates y estructuras rápidamente.

La pregunta importante no es sólo:

¿Funciona?

También debemos poder responder:

- ¿comprendo lo que hizo?
- ¿separó razonablemente las responsabilidades?
- ¿agregó dependencias necesarias?
- ¿puedo modificarlo y explicar el flujo del request?

Para analizar

Un agente genera esto:

```
@app.route("/solicitudes", methods=["POST"])
def solicitud_crear():
    # leer formulario
    # validar
    # abrir base de datos
    # ejecutar SQL
    # generar HTML
    # enviar email
    # registrar log
    # devolver respuesta
```

Puede funcionar.

¿Queremos que una única función sea responsable de todo?

Pregunta para la próxima clase

¿Cómo organizamos una aplicación que empieza a crecer?

Vamos a estudiar:

- separación de responsabilidades;
- Modelo;
- Vista;
- Controlador;
- Blueprints;
- templates con mayor profundidad.

Lo importante de hoy

Al terminar esta clase deberíamos poder explicar:

1. Qué hace Flask.
2. Qué representa una ruta.
3. Qué ocurre entre request y response.
4. Para qué sirve un template.
5. Por qué una aplicación de un solo archivo puede convertirse en un problema.
6. Por qué "funciona" no es el único criterio de calidad.

Referencias

- [Flask](#)
- [Flask Quickstart](#)
- [Flask Application Factories](#)
- [Jinja](#)
- [Backend Developer Roadmap](#)

Fin